Lecture 13 | From Euler & Hamilton to Matching

- Why do two similar-looking cycle problems have different computational difficulty?
- How does Hamiltonian Cycle lead to the Travelling Salesperson Problem?
- What makes a valid assignment of people to resources?

Today: complete the previous topic, then introduce matching

Same network, two different requirements

Inspect every connection

- A road-inspection vehicle must cover every street segment exactly once and return.
- This is an Euler circuit question.

Visit every location

- A delivery vehicle must visit each destination exactly once and return.
- This is a Hamiltonian cycle question.

Ask: Why is one problem easier to decide?

Euler circuit: a complete test

- For a connected undirected graph, an Euler circuit exists exactly when every vertex has even degree.
- Connectivity and vertex degrees can be checked in O(V + E).
- If the condition holds, Hierholzer’s algorithm constructs a circuit in O(V + E).

The condition is necessary AND sufficient.

Hamiltonian cycle: what changes?

- A Hamiltonian cycle visits every vertex exactly once before returning to the start.
- Degree ≥ 2 and absence of cut vertices are necessary conditions, but not sufficient.
- Dirac’s and Ore’s theorems give sufficient conditions, not complete tests.
- The general Hamiltonian Cycle decision problem is NP-complete.

Failure of a sufficient condition does not imply non-Hamiltonian.

Concept check: why complexity differs

- A student claims: “Both problems look for a cycle, so both should be equally easy.”
- Explain why Euler’s degree test settles its problem but Dirac’s and Ore’s conditions do not settle every Hamiltonian instance.
- Can failure of Dirac’s condition establish that a graph is non-Hamiltonian?

Discussion: local degree information vs. global vertex ordering

A structural counterexample: $K_{2,3}$

- $K_{2,3}$ is connected and has minimum degree 2.
- It has no cut vertex.
- Yet it cannot have a Hamiltonian cycle: a cycle in a bipartite graph alternates between its two parts, so a spanning cycle needs equal part sizes.

Necessary structural conditions do not guarantee Hamiltonicity.

Eulerian does not mean Hamiltonian

- Take two triangles sharing exactly one vertex (“figure-eight” graph).
- Every vertex has even degree, and the graph is connected: an Euler circuit exists.
- The shared vertex is a cut vertex: no Hamiltonian cycle exists.

Conversely, K_4 is Hamiltonian but not Eulerian (every degree is 3).

From Hamiltonian Cycle to TSP

- Hamiltonian Cycle (decision): Does a cycle visiting each vertex once exist?
- Travelling Salesperson Problem (optimization): Which tour has minimum total travel cost?
- For a complete weighted graph, a Hamiltonian tour exists; the challenge is finding the cheapest one.
- General TSP optimization is NP-hard.

Do not confuse “a valid tour exists” with “find the cheapest tour.”

Worked TSP example: four destinations

- Complete graph on A, B, C, D with symmetric costs:
- AB=10, AC=15, AD=20, BC=35, BD=25, CD=30.
- Distinct tours starting at A (ignoring reversal):
- A–B–C–D–A = 95; A–B–D–C–A = 80; A–C–B–D–A = 95.

Minimum tour cost = 80 (A–B–D–C–A, or reverse).

Why not just try every tour?

- With n destinations, the number of tours in a symmetric complete graph (fixing a start and ignoring reversal) is $(n-1)! / 2$.
- The search space grows factorially with n.
- Exact approaches include branch-and-bound and dynamic programming; larger instances often motivate heuristics.

An efficient solution for every arbitrary instance is not known.

Transition: from routing to assignment

- New challenge: allocate five students to five different FYP topics.
- Each student can select only certain eligible topics.
- No topic can be assigned to more than one student.
- Question: Can everyone receive a distinct eligible topic?

Vertices represent students and topics; edges represent eligibility.

Bipartite graph and matching

- A bipartite graph has two disjoint vertex sets; edges connect vertices across the sets.
- A matching is a set of edges with no shared endpoints.
- Maximum matching: largest possible number of matched edges.
- Perfect matching: every vertex is matched (when the two parts have equal size).

A matching that cannot be extended directly need not be maximum.